

Race Condition ինչ է — Խորը ուղեցույց

C++ Concurrency presentation

Այս ուղեցույցը մանրամասն բացատրում է race condition-ի՝ ինչ է, ինչու է linum, real orinakner (sxal + chisht lucum), inchpes haytnaberel, ev interview harcer.

1. Ի՞նչ է Race Condition-ը

Race condition-ի linum է, erb **mi qani thread miasin mutq unen (u poxum en) nuyn shared data-n**, u ardyunqy kaxvac է thread-neri kataroghakan kargic (vory-n araji hasnum).

Ardyunqy "mrcum" է (race) thread-neri mijev -> ankanxatesli

```
int counter = 0;    // shared

void increment() {
    counter++;      // NE atomic: read -> +1 -> write
}

// 2 thread -> ardyunqy karox e sxal linel
```

2. Inchu e linum — nerhatuk

`counter++` -ի iroq 3 mekusi qayl է.

1. read counter (orinak 5)
2. +1 (6)
3. write counter (6)

Erku thread miasin.

```
Thread A: read 5
Thread B: read 5    <- deř 5!
Thread A: write 6
Thread B: write 6    <- petq er 7, bayc 6 (mek increment korav)
```

Sa "lost update" e -> race condition.

3. Real orinak (SXAL kod)

```
#include <thread>
int counter = 0;

void work() {
    for (int i = 0; i < 1000000; i++)
        counter++;           // RACE CONDITION
}

int main() {
    std::thread t1(work), t2(work);
    t1.join(); t2.join();
    cout << counter;         // SPASELIQ 200000, bayc <200000 (ankanxatesli)
}
```

4. Lucum 1 — Mutex

```
#include <mutex>

int counter = 0;

std::mutex mtx;

void work() {
    for (int i = 0; i < 1000000; i++) {
        std::lock_guard<std::mutex> lock(mtx);
        counter++;           // miayn mek thread miangamic
    }
}

// hima counter == 200000 (chisht)
```

5. Lucum 2 — Atomic (aveli arag)

```
#include <atomic>

std::atomic<int> counter{0};

void work() {
    for (int i = 0; i < 100000; i++)
        counter++;           // ATOMIC -- anabajaneli, chka race
}

// counter == 200000, ařanc mutex-i (aveli arag payer kařucacqneri hamar)
```

Pogr operacia-neri (counter++) hamar `atomic` -y aveli arag e qan mutex. (Tes atomic presentation.)

6. Data Race vs Race Condition

Data Race: erku thread synchronization-ic ařanc nuyn memory mutq
(ařnvazn meky write) -> UNDEFINED BEHAVIOR

Race Condition: ardyungy kaxvac e thread-neri kargic (logikakan sxal)

Data race-y misht sxal e (UB); race condition-y aveli layn haskacutyun (logikakan bug)。

7. Inchpes haytnaberel

```
ThreadSanitizer (TSan): g++ -fsanitize=thread ...  
-> haytnaberum data race-nery runtime-um  
Helgrind (Valgrind):    valgrind --tool=helgrind  
Code review:           shared mutable data + synchronization-i bacakayutyun
```

8. Xusapelu dzevery

1. Mutex / lock_guard -> critical section
2. Atomic -> poqr operacia (counter, flag)
3. Immutable data -> chka poxum -> chka race
4. Thread-local data -> amen thread ir copy-n
5. Chxaʁnel shared mutable state (aveli lav design)

9. Interview harcer & patasxanner

Race condition inch e?

Mi qani thread miasin poxum en nuyn shared data; ardyunqy kaxvac thread-neri kargic -> ankanxatesli.

Inchu counter++ race e?

Ne atomic -- read/+1/write 3 qayl; thread-ner xaʁnvum -> lost update.

Inchpes lucel?

Mutex (lock_guard) kam atomic; immutable/thread-local data.

Data race vs race condition?

Data race sync-ic arfanc nuyn memory (write) -> UB. Race condition logikakan, ardyunq kargic kaxvac.

Inchpes haytnaberel?

ThreadSanitizer (-fsanitize=thread), Helgrind, code review.

Mutex vs atomic race-i hamar?

Poqr operacia (counter/flag) -> atomic (arag); bardz critical section -> mutex.

Cheat Sheet

RACE CONDITION = mi qani thread nuyn shared data poxum -> ardyunq kargic kaxvac

INCHU: counter++ ne atomic (read/+1/write) -> lost update

LUCUM:

mutex / lock_guard -> critical section

atomic -> poqr operacia (arag)

immutable / thread-local data

DATA RACE (UB) vs RACE CONDITION (logikakan bug)

HAYTNABEREL: ThreadSanitizer (-fsanitize=thread) | Helgrind